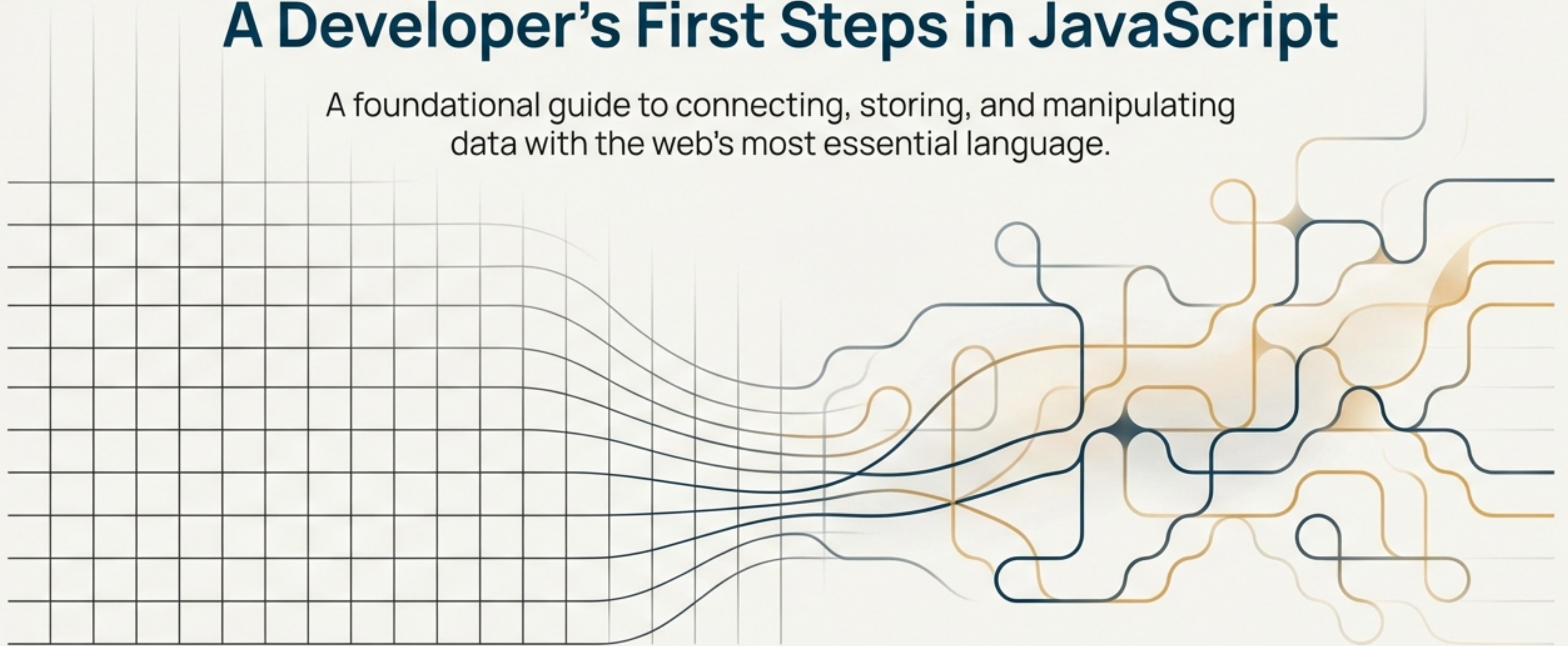


From Setup to Scripting: A Developer's First Steps in JavaScript

A foundational guide to connecting, storing, and manipulating data with the web's most essential language.



What You Will Master



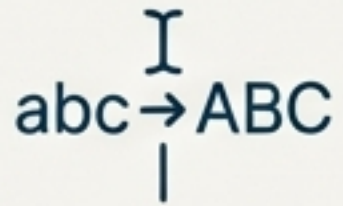
Connect & Execute: Use the `<script>` tag to efficiently add JavaScript to your pages, ensuring fast, non-blocking performance.



Store & Reference: Declare variables using `let` and `const` to hold strings, numbers, and booleans—the building blocks of any program.



Calculate & Compute: Perform arithmetic operations with numbers, understanding the rules of operator precedence.



Format & Transform: Manipulate text using powerful string methods to query, clean, and combine data.



Debug & Inspect: Use the browser's `JavaScript Console` as your essential tool for viewing output and diagnosing errors.

The First Connection: Introducing the `<script>` Tag

The `<script>` tag is the bridge between your HTML structure and your JavaScript logic. There are two primary ways to use it.

Inline JavaScript

Placing your code directly within the HTML file. Best for very small, page-specific scripts.

```
...  
<body>  
  <!-- HTML content -->  
  <script>  
    console.log("Hello from web page.");  
  </script>  
</body>  
</html>
```

↪ HTML Always place scripts just before the closing `</body>` tag to ensure HTML content loads first.

External JavaScript

Linking to a separate `.js` file. This is the recommended best practice for organization and performance.

```
<head>  
  ...  
  <script defer src="script.js"></script>  
</head>  
...
```

🕒 The `defer` attribute is crucial. It tells the browser to load the script without blocking the HTML from rendering.

The Professional Standard: External Scripts for Performance and Clarity

Linking to external `.js` files is superior for three key reasons.



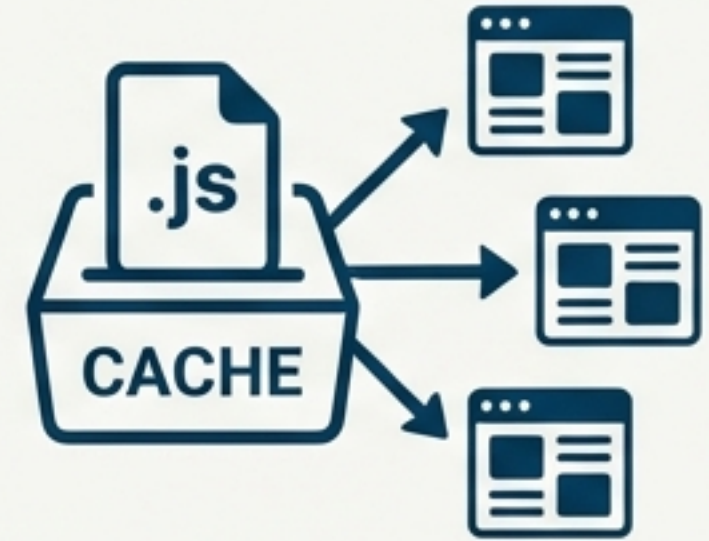
1. Separation of Concerns

Keeps your HTML (structure) and JavaScript (behavior) clean and separate, making code easier to read and maintain.



2. Faster Page Loads

The **defer** attribute creates a *non-blocking* script. This means the user sees your page content immediately, without waiting for JavaScript to download and execute. This directly improves user experience.



3. Efficient Caching

On multi-page sites, browsers can cache the external **.js** file. After the first page visit, the script loads instantly on subsequent pages, speeding up the entire site.

Your Command Center: The JavaScript Console

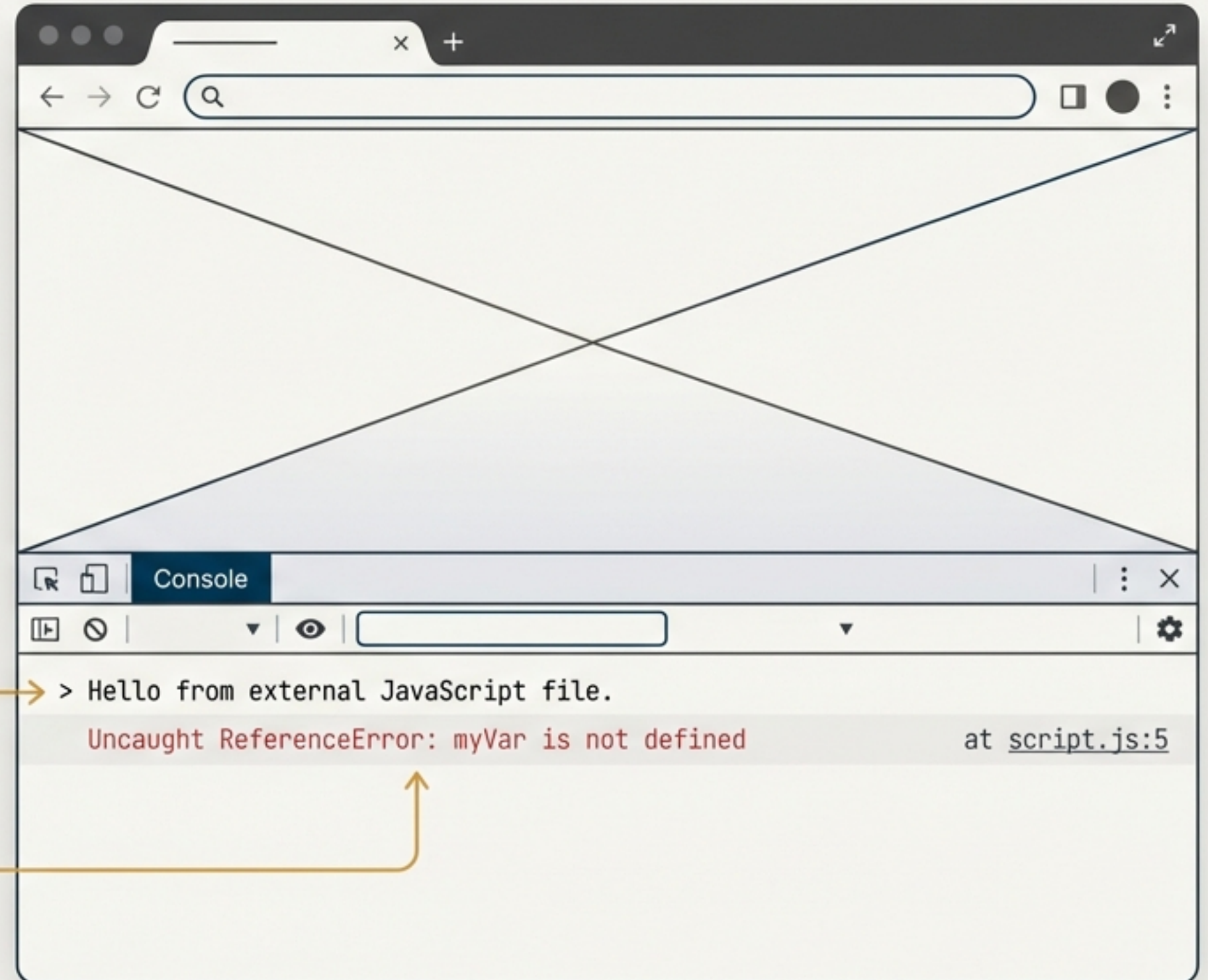
Your browser's Developer Tools (DevTools) contain a powerful console, an interactive environment to test code, inspect values, and see error messages.

How to Open DevTools

- Windows: `Ctrl` + `Shift` + `i`
- Mac: `⌘` + `⌥` + `i`
- Or right-click on the page and select 'Inspect'

What You'll Use It For

- **Viewing Output**: Use `console.log()` in your code to print values and messages, helping you understand what your script is doing.
- **Spotting Errors**: The Console will automatically report any errors in your code, including the file name and line number, making debugging much faster.



Storing Information: The Role of Variables

Think of a variable as a labeled container or “storage box” for a value that might change each time the script runs. The name of the container stays the same, but its contents can be updated.

const (Constant)



Use Case: For values that should *never* change after being set.

Analogy: A person’s date of birth or a fixed tax rate.

```
const taxRate = 0.20;
```

let



Use Case: For values that you expect to *reassign* or update later in the code.

Analogy: A player’s score in a game or a customer’s current address.

```
let playerScore = 100;
```

Historical Note: Older JavaScript used the `var` keyword. This is no longer recommended. Use `let` and `const`.

The Art of Naming: Rules and Conventions

Clear variable names make your code readable and maintainable. Follow these industry-standard practices.



Use camelCase

The first word is lowercase, and each subsequent word is capitalized.

```
let userFirstName; ✓
```

```
const accountBalance; ✓
```

No Spaces or Dashes

These will cause errors.

```
let SubTotal; // Invalid ✗
```

```
const Product-Name; // Invalid ✗
```

Names are Case-Sensitive

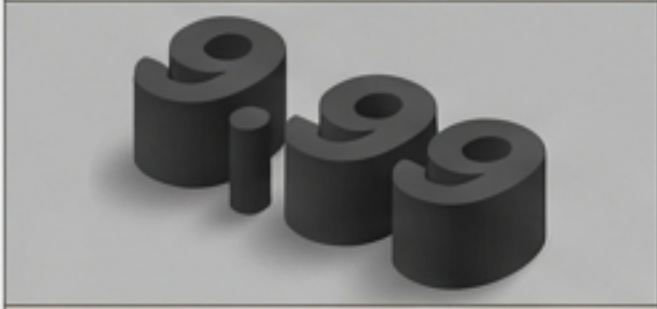
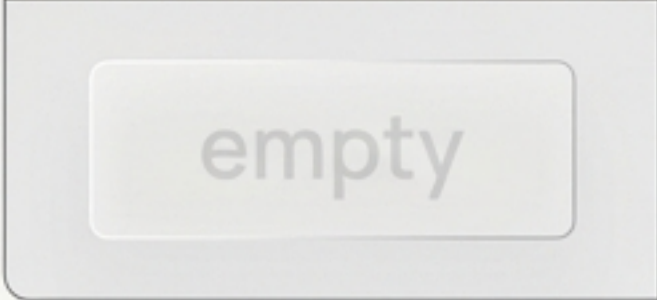
`firstName` and `FirstName` are two different variables.

Be Unique

Don't declare two different variables with the same name within the same scope.

The Building Blocks: Core Data Types in JavaScript

JavaScript is a *loosely-typed* language. You don't specify the data type; JavaScript infers it from the value you assign. The same variable can hold different types at different times.

	Data Type	Description	Example
	string	A sequence of characters (text). Must be enclosed in quotes (`\"`, `'`) or backticks (` ` `).	<pre>const name = "Sheila Murphy";</pre>
	number	Any numeric value, including integers and decimals (floats). No quotes.	<pre>let productPrice = 9.99;</pre>
	boolean	Represents a logical value of either `true` or `false`. Often used for comparisons.	<pre>const isLoggedIn = true;</pre>
	null	Intentionally represents 'no value.' It signifies a deliberate absence of data.	<pre>let testScore = null;</pre>

Pro-Tip: You can check a variable's type at any time with the `typeof` operator:

```
console.log(typeof productPrice); // Outputs: "number"
```


An Elegant Combination: Mixing Variables and Text with Template Literals

You often need to construct a string that includes the values from your variables.

The Modern Solution: Template Literals

- Enclose the entire string in backticks (`).
- Embed variables directly into the string using the `\${variableName}` syntax.

```
const customerFirstName = "Marie";
const accBalance = 10;

// The old way (concatenation with '+')
// console.log("Hi " + customerFirstName + ". Your balance is €" + accBalance);

// The modern way (template literals)
console.log(`Hi ${customerFirstName}. Your balance is €${accBalance}.`);
// Outputs: Hi Marie. Your balance is €10.
```

Template literals are cleaner, more readable, and less error-prone than traditional string concatenation.

The Language of Logic: Working with Numbers

Numeric variables can be integers (25) or floating-point numbers (9.99). Arithmetic is performed with standard operators.

Arithmetic Operators Table

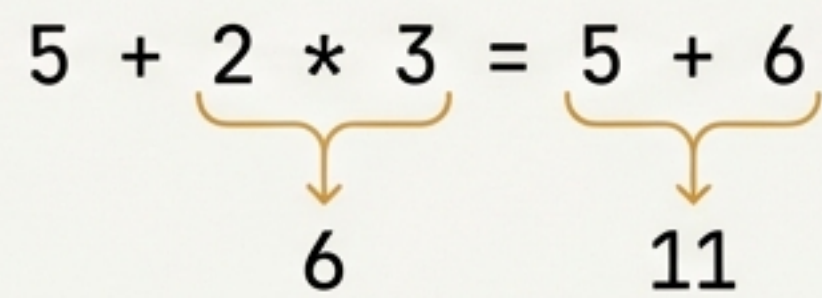
Operation	Operator	Example	Result
Addition	+	12 + 8	20
Subtraction	-	12 - 8	4
Multiplication	*	12 * 8	96
Division	/	12 / 8	1.5

Critical Rule: Operator Precedence

JavaScript follows standard mathematical rules: multiplication and division are performed *before* addition and subtraction.

Example 1

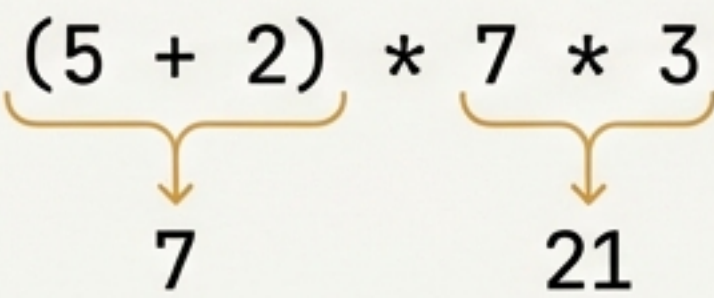
```
console.log(5 + 2 * 3);
```



Example 2

To control the order, use parentheses ().

```
console.log((5 + 2) * 3);
```

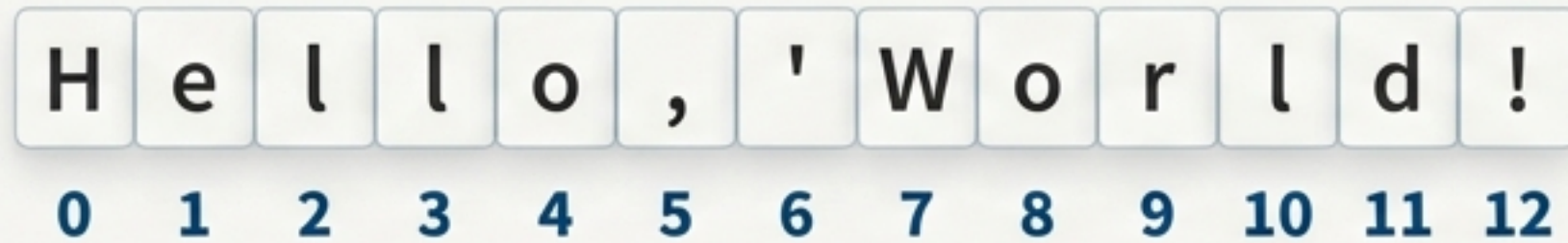


The Language of Text: Understanding String Structure

A string is an ordered sequence of characters. JavaScript provides built-in properties and notation to interact with this structure.

Zero-Based Indexing

Each character has an index number, starting from `0` for the first character. You can access any character using square bracket notation `[n]`.



```
let myStr = "Hello, World!";
```

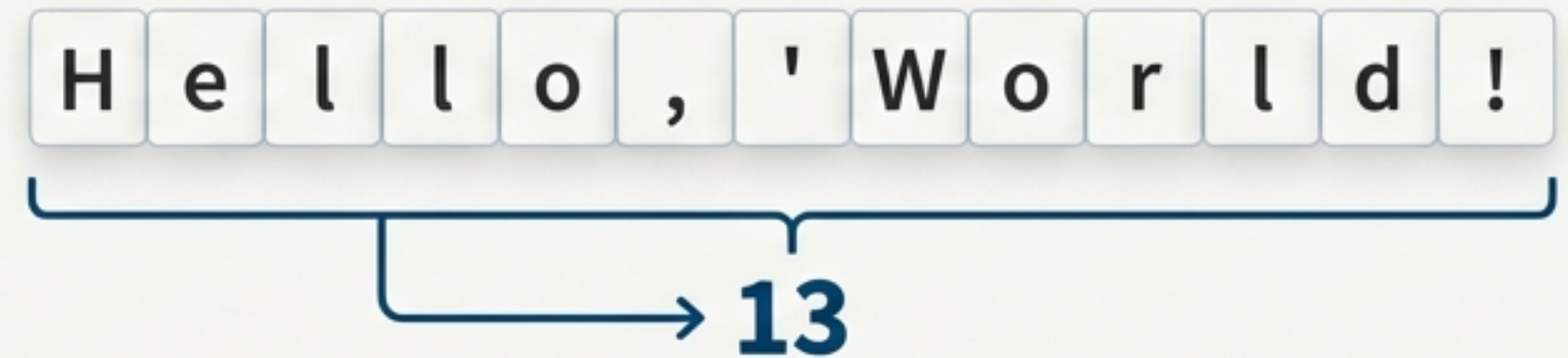
```
console.log(myStr[0]); // Outputs: H
```

```
console.log(myStr[7]); // Outputs: W
```

```
console.log(myStr[12]); // Outputs: !
```

The .length Property

Every string has a `.length` property that returns the total number of characters it contains.



```
let myStr = "Hello, World!";
```

```
console.log(myStr.length); // Outputs: 13
```

The length is always one greater than the highest index number (`13` vs `12`).

Powerful String Methods: Your Toolkit for Text Manipulation

Methods are built-in actions you can perform on a string.
They are always followed by parentheses `()`.

Querying & Checking (Return `true`/`false` or an `index`)



```
.includes("World")
```

Does the string contain this substring?



```
.startsWith("Hello")
```

Does it begin with this?



```
.endsWith("!")
```

Does it end with this?



```
.indexOf("World")
```

Returns the starting index of the substring (or `-1` if not found).

Transforming (Return a `new, modified string`)

```
.trim()
```

Removes whitespace from the beginning and end.

```
.toUpperCase()
```

Converts the entire string to uppercase.

```
.toLowerCase()
```

Converts the entire string to lowercase.

Extracting & Replacing (Return a `new, modified string`)

```
.slice(0, 10)
```

Extracts a portion of the string.

```
.replace("o", "0")
```

Replaces the *first* occurrence.

```
.replaceAll("o", "0")
```

Replaces **all** occurrences.

A Critical Concept: String Methods Do Not Change the Original

In JavaScript, strings are *immutable*. When you use a string method like `.toUpperCase()`, it does **not** alter the original string. Instead, it **returns a new string** with the changes applied.

Demonstration (The "Wrong" Way)

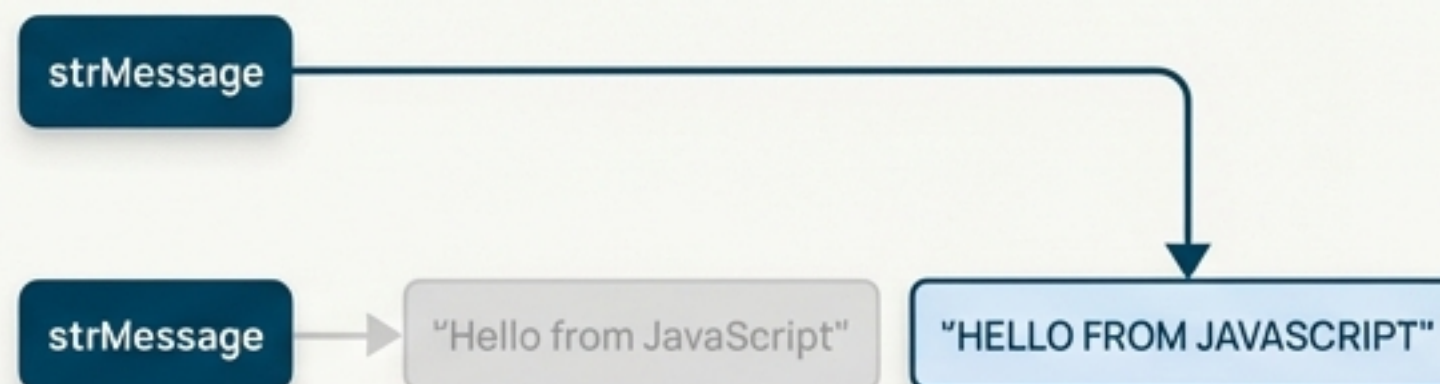
```
let strMessage = "Hello from JavaScript";  
console.log(strMessage); // Outputs: "Hello from JavaScript"  
  
// This line runs, creates a new uppercase string, but does nothing with it  
strMessage.toUpperCase();  
  
console.log(strMessage); // Still outputs: "Hello from JavaScript"
```



The **Correct Way** to Modify a String

To make your changes permanent, you must **reassign** the result of the method back to a variable (either the original or a new one).

```
// Assign the new, transformed string back to the original variable  
strMessage = strMessage.toUpperCase();  
  
console.log(strMessage); // NOW outputs: "HELLO FROM JAVASCRIPT"
```



Your Journey Forward

You Can Now...

- ✓ Confidently connect JavaScript to any web page for optimal performance.
- ✓ Use variables to store and manage different types of data.
- ✓ Perform calculations and build complex strings with ease.
- ✓ Manipulate text and inspect your work using the browser console.

The Road Ahead

Foundation — where we are now

You have mastered the fundamental **'nouns'** (data) and **'verbs'** (operators/methods) of JavaScript.

Your next steps involve learning how to create reusable blocks of code (**Functions**) and how to make your scripts react to different conditions (**Making Decisions** with ``if`` statements).

This foundation is the key to unlocking true interactivity.

